

Agentic Loops in Claude

Exam quick reference

60 questions	120 minutes	720 / 1000 pass	\$99 USD
<i>Questions</i>	<i>Duration</i>	<i>Pass score</i>	<i>Fee</i>

Domain weights

Agentic architecture & orchestration	27%
Claude Code configuration & workflows	20%
Prompt engineering & structured output	20%
Tool design & MCP integration	18%
Context management & reliability	15%

1. What is an agentic loop?

An agentic loop is the core pattern behind any Claude-powered autonomous system. Instead of a single prompt → response, Claude operates in a cycle: it receives a goal, decides what action to take, executes that action, observes the result, and repeats until the goal is complete or a stopping condition is hit. Think of it as Claude acting like a developer at a terminal — not just answering, but *doing*.

The five stages

1	Goal / user prompt
2	Plan next action
3	Execute tool / action
4	Observe result
5	Goal complete?

The loop repeats from step 5 back to step 2 until the goal is met or a budget/failure condition triggers an exit.

2. Key design decisions (exam-critical)

Stopping conditions — design these first

- Success condition — the goal is detectably met (e.g. tests pass, file written).
- Budget condition — max steps, max tokens, or max time reached.
- Failure condition — too many errors, or confidence drops below threshold.
- All three must be defined explicitly before deploying any agentic loop.

Tool call granularity

- Each tool should do one thing only.
- A multi-purpose tool creates ambiguous failure states — you won't know which part failed.
- Keep tools narrow and composable.

Error handling inside the loop

- Retry with backoff — same tool, wait and try again (for transient errors like rate limits).
- Fallback action — different tool or strategy (for permanent failures like a missing resource).
- Always cap retries (typically 2–3 attempts) before switching strategy.

Human-in-the-loop checkpoints

- For irreversible actions (deleting data, sending emails, purchases) — pause and ask for confirmation.
- This is the minimal footprint principle: do as little as possible autonomously.
- Irreversible = human approval required. Every time.

Context window as the loop's memory

- Every tool call and result lives in Claude's context window.
- Long loops gradually fill up the context — older tokens get truncated from the start.
- Strategy: summarise earlier steps and drop stale information proactively.
- This is called a handoff pattern — critical for Week 3 context management.

3. Common failure modes

Failure	Cause	Fix
Infinite loop	No stopping condition	Set max_steps budget
Hallucinated tool output	Claude guesses result without calling tool	Always route through real tool execution
Context overflow	Long loop fills the window	Summarise + compress older steps
Cascading errors	One failed step breaks all downstream steps	Validate output of each step before proceeding
Over-permissioned actions	Tool can do more than needed	Scope tools to minimum required permissions

4. Quick recall — 3 things to remember

- 1 Plan → Act → Observe is the core loop.**
Claude reasons before every action — it doesn't just fire tools blindly.
- 2 Stopping conditions are non-negotiable.**
Max steps + success check + failure check. Always define all three.
- 3 Minimal footprint principle.**
Irreversible actions must pause and require explicit human approval before execution.

5. Practice questions & answers

These three questions were answered during today's session. The correct answer is marked with ✓. Review the explanations carefully — these concepts map directly to exam scenarios.

Q1. A Claude agent is tasked with autonomously refactoring a large codebase. It has tools for reading files, writing files, and running tests. After 47 steps, the context window is nearly full and the task isn't complete. What is the BEST architectural decision?

A) Increase the model's max_tokens parameter to allow more context

✓ **B) Summarise completed steps and inject a compressed state summary at the start of the next loop iteration**

C) Restart the loop from scratch with the original goal

D) Allow the loop to continue — Claude will automatically drop irrelevant context

Explanation:

- B is correct: When the context window fills mid-loop, the best pattern is to summarise completed steps into a compressed state object and carry only that forward. This is the handoff pattern — preserving progress without losing the thread.
- A is wrong: max_tokens controls output length, not context window size.
- C is wrong: Restarting loses all 47 steps of progress — wasteful and incorrect.
- D is wrong: Claude does NOT automatically drop context intelligently. When the window fills, older tokens are truncated from the start — losing your original goal and early reasoning.

Q2. You are designing a Claude agent that autonomously sends customer refund emails after verifying eligibility. A colleague suggests the agent should send emails immediately once eligibility is confirmed, with no human review step. What principle does this violate, and what should you do instead?

A) It violates tool granularity — split the verify and send tools into separate agents

✓ **B) It violates the minimal footprint principle — pause before irreversible actions and require human approval**

C) It violates session management — the agent needs a handoff pattern before sending

D) Nothing is wrong — fully autonomous email sending is fine for production agents

Explanation:

- B is correct: Sending an email is irreversible. The minimal footprint principle states that Claude agents must pause and request human confirmation before any action that cannot be undone.
- A is wrong: Tool granularity is about keeping tools narrow — not the issue here.
- C is wrong: Handoff patterns address context window management, not irreversible actions.
- D is wrong: Fully autonomous irreversible actions without a human checkpoint is an anti-pattern for production systems, especially customer-facing ones.

Q3. A Claude agent is running a research pipeline. At step 12, a web search tool returns a 404 error. The agent retries the same URL three times and fails each time. What is the correct error handling strategy?

A) Abort the entire loop and report failure to the user

B) Retry indefinitely with exponential backoff until the URL responds

✓ **C) Switch to a fallback action — try an alternative search query or different tool — and continue the loop**

D) Skip the step entirely and proceed, assuming the missing data is non-critical

Explanation:

- C is correct: After repeated failures on the same tool call, switch to a fallback action — a different query, source, or tool. This is the distinction between retry with backoff (transient errors) and fallback action (permanent failures like a dead URL).
- A is wrong: Aborting the entire pipeline over one failed URL is disproportionate.
- B is wrong: Retrying indefinitely creates infinite loops. Cap retries at 2–3 before switching strategy.
- D is wrong: Silently skipping steps without logging is dangerous — if the data was critical, downstream steps produce wrong results with no traceable cause.

Day 1 scorecard

3 / 3

Perfect score!

Agentic loop anatomy	✓ Covered
Stopping conditions	✓ Covered
Context overflow handling	✓ Covered
Minimal footprint principle	✓ Covered
Error handling strategies	✓ Covered

Next up: Day 2

Multi-agent patterns — orchestrators, subagents, and hub-and-spoke models.